



GROWMORE GROUP OF INSTITUTE

UNIT – 2

DIMPI GOR

(ASSISTANT PROFESSOR OF GROW MORE BCA & PGDCA)

Process Management in OS

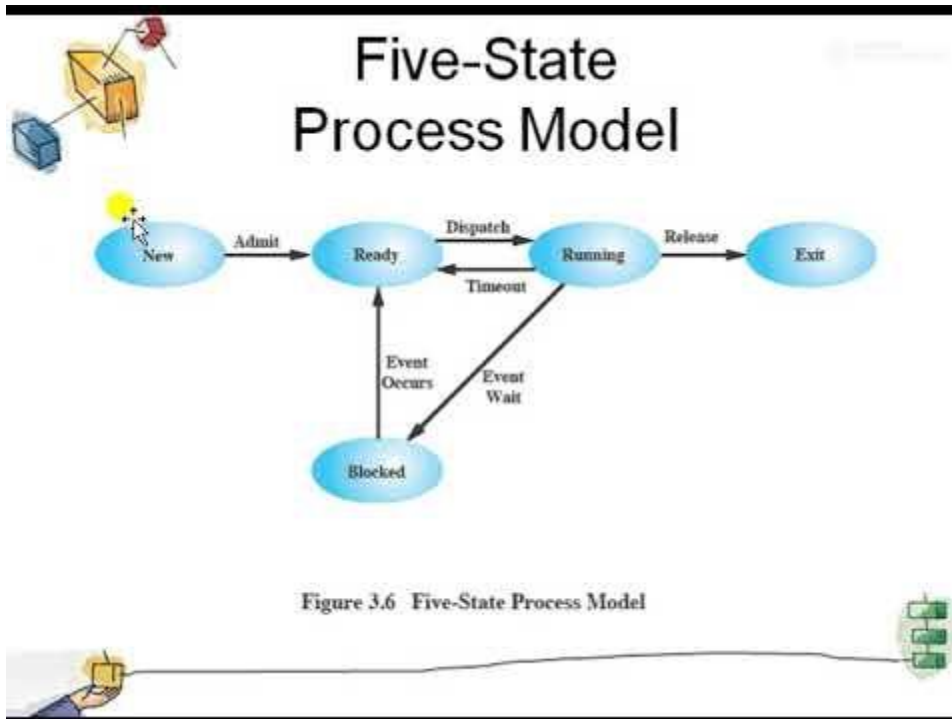
Introduction to process:

A Program does nothing unless its instructions are executed by a CPU. A program in execution is called a process. In order to accomplish its task, process needs the computer resources.

There may exist more than one process in the system which may require the same resource at the same time. Therefore, the operating system has to manage all the processes and the resources in a convenient and efficient way.

Process States in OS

We'll learn about 'Process States' and the many states processes that occur throughout the lifespan of a process in this post. The states that the process can have differed from one Operating System to the next. However, just a few frequent states are depicted in the diagram.



Process States in OS

We'll learn about 'Process States' and the many states processes that occur throughout the lifespan of a process in this post. The states that the process can have differed from one Operating System to the next. However, just a few frequent states are depicted in the diagram.

List of States of the Process

1. New State-

- A process is said to be in new state when a program present in the secondary memory is initiated for execution.

2. Ready State-

- A process moves from new state to ready state after it is loaded into the main memory and is ready for execution.
- In ready state, the process waits for its execution by the processor.
- In multiprogramming environment, many processes may be present in the ready state.

3. Run State-

- A process moves from ready state to run state after it is assigned the CPU for execution.

4. Terminate State-

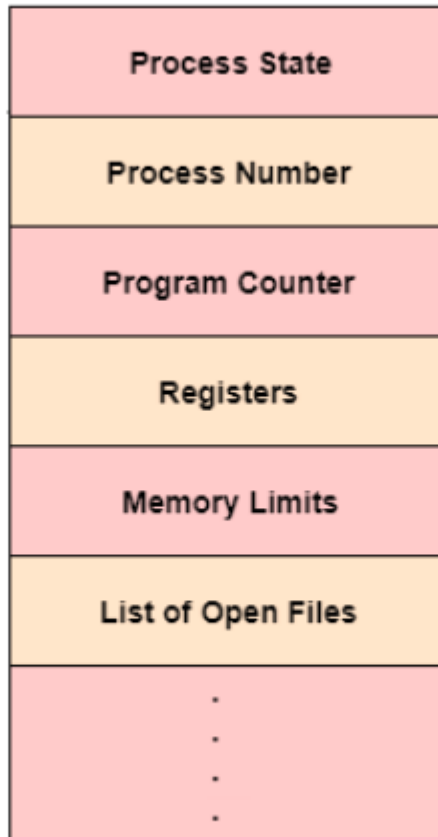
- A process moves from run state to terminate state after its execution is completed.
- After entering the terminate state, context (PCB) of the process is deleted by the operating system.

5. Block Or Wait State-

- A process moves from run state to block or wait state if it requires an I/O operation or some blocked resource during its execution.
- After the I/O operation gets completed or resource becomes available, the process moves to the ready state.

Structure of the Process Control Block (PCB)

The process control stores many data items that are needed for efficient process management. Some of these data items are explained with the help of the given diagram –



Process Control Block (PCB)

The following are the data items –

Process State

This specifies the process state i.e. new, ready, running, waiting or terminated.

Process Number

This shows the number of the particular process.

Program Counter

This contains the address of the next instruction that needs to be executed in the process.

Registers

This specifies the registers that are used by the process. They may include accumulators, index registers, stack pointers, general purpose registers etc.

List of Open Files

These are the different files that are associated with the process

CPU Scheduling Information

The process priority, pointers to scheduling queues etc. is the CPU scheduling information that is contained in the PCB. This may also include any other scheduling parameters.

Memory Management Information

The memory management information includes the page tables or the segment tables depending on the memory system used. It also contains the value of the base registers, limit registers etc.

I/O Status Information

This information includes the list of I/O devices used by the process, the list of files etc.

Accounting information

The time limits, account numbers, amount of CPU used, process numbers etc. are all a part of the PCB accounting information.

Location of the Process Control Block

The process control block is kept in a memory area that is protected from the normal user access. This is done because it contains important process information. Some of the operating systems place the PCB at the beginning of the kernel stack for the process as it is a safe location.

Scheduling Queue

The processes that are entering into the system are stored in the **Job Queue**. Suppose if the processes are in the Ready state are generally placed in the **Ready Queue**.

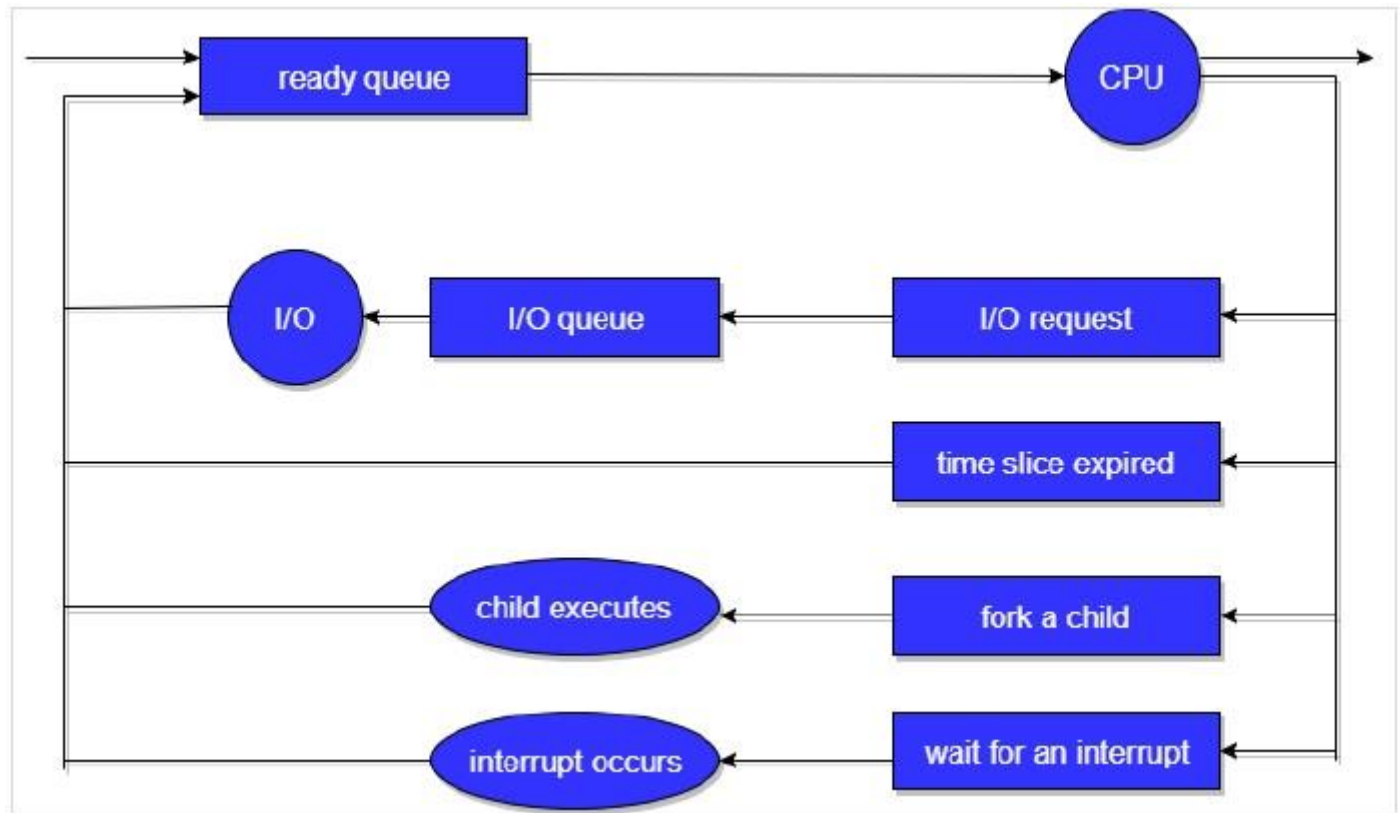
The processes waiting for a device are placed in **Device Queues**. There are unique device queues which are available for every I/O device.

First place a new process in the **Ready queue** and then it waits in the ready queue till it is selected for execution.

Once the process is assigned to the CPU and is executing, any one of the following events occur

—

- The process issue an I/O request, and then placed in the I/O queue.
- The process may create a new sub process and wait for termination.
- The process may be removed forcibly from the CPU, which is an interrupt, and it is put back in the ready queue.



In the first two cases, the process switches from the waiting state to the ready state, and then puts it back in the ready queue. A process continues this cycle till it terminates, at which time it is removed from all queues and has its PCB and resources deallocated.

Types of Schedulers

There are **three** types of schedulers available which are as follows –

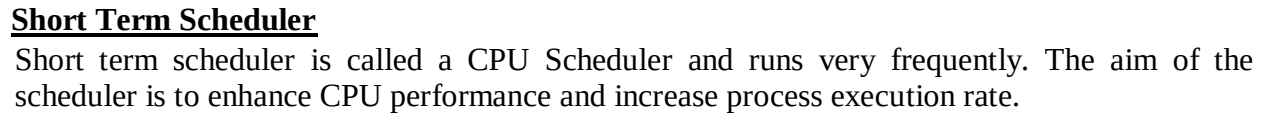
Long Term Scheduler

Long term scheduling is performed when a new process is created, if the number of ready processes in the ready queue becomes very high. Then, there is an overhead on the operating system, for maintaining long lists, containing switching and dispatching increases. Therefore, allowing only a limited number of processes into the ready queue, the long term scheduler manages this.

Long term scheduler runs less frequently. It decides which program must get into the job queue. From the job queue, the job processor selects processes and loads them into the memory for execution.

The main aim of the Job Scheduler is to maintain a good degree of Multiprogramming. The degree of Multiprogramming means the average rate of process creation is equal to the average departure rate of processes from the execution memory.

The diagram of long term and short term scheduler is as follows –



This type of scheduling removes the processes from memory and thus reduces the degree of multiprogramming. Later, the process is reintroduced into memory and its execution is continued where it left off. This is called **swapping**. The process is swapped out, and is later swapped in, by the medium term scheduler.

The diagram of medium term scheduler is as follows –



FCFS Scheduling Algorithm: What is, Example Program

First Come First Serve (FCFS) is an operating system scheduling algorithm that automatically executes queued requests and processes in order of their arrival. It is the easiest and simplest CPU scheduling algorithm. In this type of algorithm, processes which requests the CPU first get the CPU allocation first. This is managed with a FIFO queue. The full form of FCFS is First Come First Serve.

Characteristics of FCFS method

- Jobs are always executed on a first-come, first-serve basis.
- It is easy to implement and use.
- This method is poor in performance, and the general wait time is quite high.

Example of FCFS scheduling

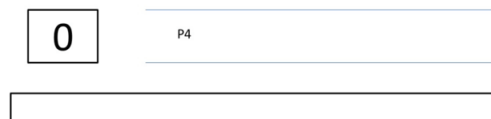
- A real-life example of the FCFS method is buying a movie ticket on the ticket counter. In this scheduling algorithm, a person is served according to the queue manner. The person who arrives first in the queue first buys the ticket and then the next one.

How FCFS Works? Calculating Average Waiting Time

- Here is an example of five processes arriving at different times. Each process has a different burst time.

Process	Burst time	Arrival time
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4

Step 0) The process begins with P4 which has arrival time 0



BCA-303 (Operating System)

Step 1) At time=1, P3 arrives. P4 is still executing. Hence, P3 is kept in a queue.

Process	Burst time	Arrival time
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4

1

P3

P4

Step 2) At time= 2, P1 arrives which is kept in the queue.

Process	Burst time	Arrival time
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4

2

P3 P1

P4

Step 3) At time=3, P4 process completes its execution.

3

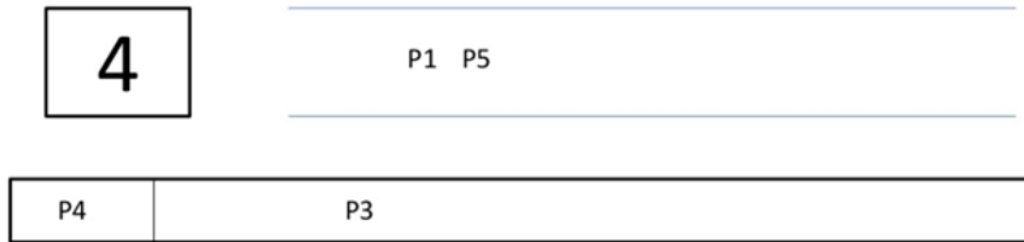
P3 P1

P4

BCA-303 (Operating System)

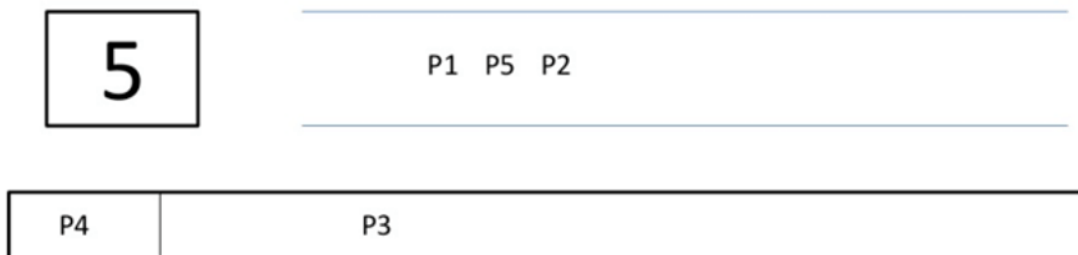
Step 4) At time=4, P3, which is first in the queue, starts execution.

Process	Burst time	Arrival time
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4

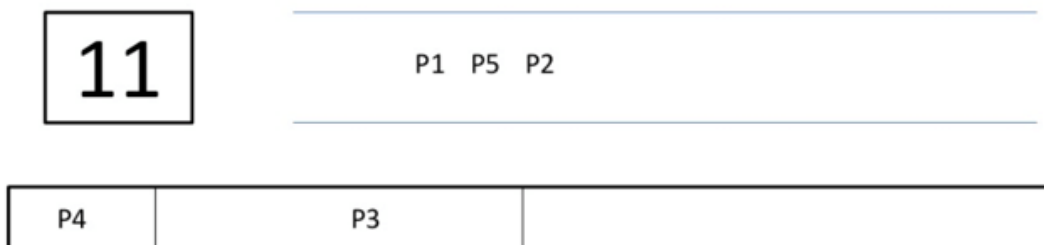


Step 5) At time =5, P2 arrives, and it is kept in a queue.

Process	Burst time	Arrival time
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4

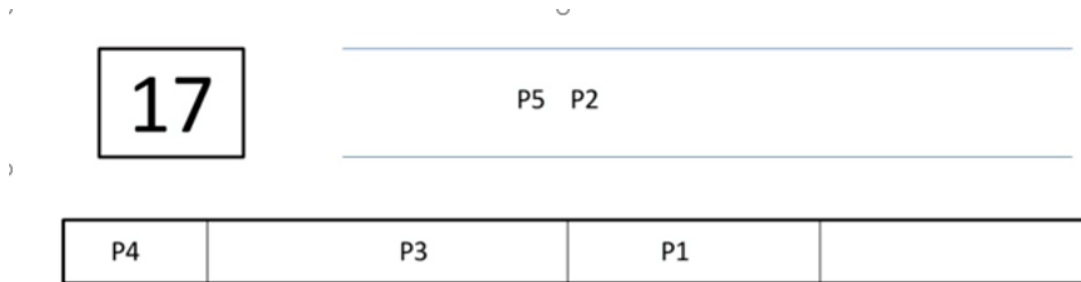


Step 6) At time 11, P3 completes its execution.

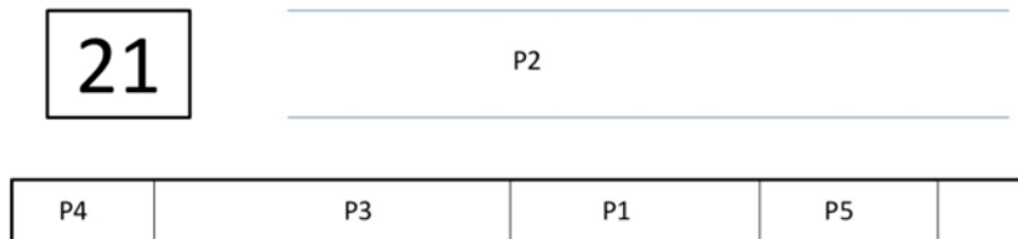


BCA-303 (Operating System)

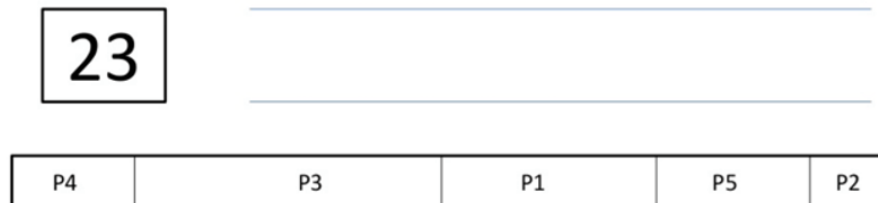
Step 7) At time=11, P1 starts execution. It has a burst time of 6. It completes execution at time interval 17



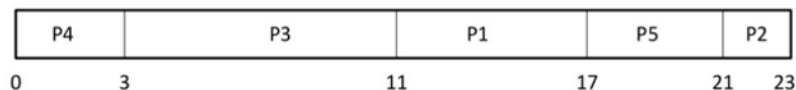
Step 8) At time=17, P5 starts execution. It has a burst time of 4. It completes execution at time=21



Step 9) At time=21, P2 starts execution. It has a burst time of 2. It completes execution at time interval 23



Step 10) Let's calculate the average waiting time for above example.



Waiting time = Start time - Arrival time

$$P4 = 0 - 0 = 0$$

$$P3 = 3 - 1 = 2$$

$$P1 = 11 - 2 = 9$$

$$P5 = 17 - 4 = 13$$

$$P2 = 21 - 5 = 16$$

Average Waiting Time

$$\frac{0+2+9+13+16}{5}$$

$$= 40/5 = 8$$

Advantages of FCFS

- The simplest form of a CPU scheduling algorithm
- Easy to program
- First come first served

Disadvantages of FCFS

- The Average Waiting Time is high.
- Short processes that are at the back of the queue have to wait for the long process at the front to finish.
- Not an ideal technique for time-sharing systems.
- Because of its simplicity, FCFS is not very efficient.

Shortest Job First (SJF) Scheduling

Till now, we were scheduling the processes according to their arrival time (in FCFS scheduling). However, SJF scheduling algorithm, schedules the processes according to their burst time.

In SJF scheduling, the process with the lowest burst time, among the list of available processes in the ready queue, is going to be scheduled next.

However, it is very difficult to predict the burst time needed for a process hence this algorithm is very difficult to implement in the system.

Example

In the following example, there are five jobs named as P1, P2, P3, P4 and P5. Their arrival time and burst time are given in the table below.

PID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
1	1	7	8	7	0
2	3	3	13	10	7
3	6	2	10	4	2
4	7	10	31	24	14
5	9	8	21	12	4

Since, No Process arrives at time 0 hence; there will be an empty slot in the **Gantt chart** from time 0 to 1 (the time at which the first process arrives).

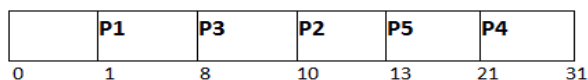
According to the algorithm, the OS schedules the process which is having the lowest burst time among the available processes in the ready queue.

Till now, we have only one process in the ready queue hence the scheduler will schedule this to the processor no matter what is its burst time.

This will be executed till 8 units of time. Till then we have three more processes arrived in the ready queue hence the scheduler will choose the process with the lowest burst time.

Among the processes given in the table, P3 will be executed next since it is having the lowest burst time among all the available processes.

So that's how the procedure will go on in **shortest job first (SJF)** scheduling algorithm.



$$\text{Avg Waiting Time} = 27/5$$

Advantages of SJF

1. Maximum throughput
2. Minimum average waiting and turnaround time

Disadvantages of SJF

1. May suffer with the problem of starvation
2. It is not implementable because the exact Burst time for a process can't be known in advance.

Round Robin scheduling

Round Robin is a [CPU scheduling algorithm](#) where each process is assigned a fixed time slot in a cyclic way. It is basically the [preemptive](#) version of [First come First Serve CPU Scheduling algorithm](#).

- Round Robin CPU Algorithm generally focuses on Time Sharing technique.
- The period of time for which a process or job is allowed to run in a pre-emptive method is called time **quantum**.
- Each process or job present in the ready queue is assigned the CPU for that time quantum, if the execution of the process is completed during that time then the process will **end** else the process will go back to the **waiting table** and wait for its next turn to complete the execution.

Characteristics of Round Robin CPU Scheduling Algorithm:

- It is simple, easy to implement, and starvation-free as all processes get fair share of CPU.
- One of the most commonly used technique in CPU scheduling as a core.
- It is preemptive as processes are assigned CPU only for a fixed slice of time at most.
- The disadvantage of it is more overhead of context switching.

Advantages of Round Robin CPU Scheduling Algorithm:

- A round-robin scheduler generally employs time-sharing, giving each job a time slot or quantum.
- While performing a round-robin scheduling, a particular time quantum is allotted to different jobs.
- Each process gets a chance to reschedule after a particular quantum time in this scheduling.

Disadvantages of Round Robin CPU Scheduling Algorithm:

- There is Larger waiting time and Response time.
- There is Low throughput.
- There is Context Switches.

Examples to show working of Round Robin Scheduling Algorithm:

Example: Consider the following table of arrival time and burst time for four processes P1, P2 and P3 and given **Time Quantum = 2**

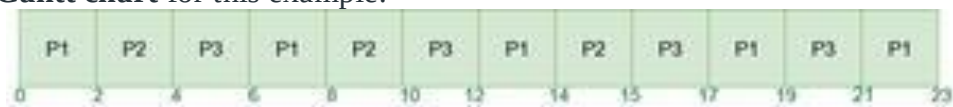
Process	Burst Time	Arrival Time
---------	------------	--------------

P1	10 ms	0 ms
----	-------	------

P2	5 ms	0 ms
----	------	------

P3	8 ms	0 ms
----	------	------

Similarly, **Gantt chart** for this example:



Gantt chart for example 2

Now, lets calculate average [waiting time and turn around](#) time:

Processes	AT	BT	CT	TAT	WT
P1	0	10	23	$23-0 = 23$	$23-10 = 13$
P2	0	5	15	$15-0 = 15$	$15-5 = 10$
P3	0	8	21	$21-0 = 21$	$21-8 = 13$

Total Turn Around Time = 59 ms

So, **Average Turn Around Time** = $59/3 = 19.667$ ms

And, Total Waiting Time = 36 ms

So, **Average Waiting Time** = $36/3 = 12.00$ ms

Thread in Operating System

What is a Thread?

A thread is a path of execution within a process. A process can contain multiple threads.

Why Multithreading?

A thread is also known as lightweight process. The idea is to achieve parallelism by dividing a process into multiple threads. For example, in a browser, multiple tabs can be different threads. MS Word uses multiple threads: one thread to format the text, another thread to process inputs, etc. More advantages of multithreading are discussed below

Advantages of Thread over Process

1. **Responsiveness:** If the process is divided into multiple threads, if one thread completes its execution, then its output can be immediately returned.

2. **Faster context switch:** Context switch time between threads is lower compared to process context switch. Process context switching requires more overhead from the CPU.

4. **Resource sharing:** Resources like code, data, and files can be shared among all threads within a process.

5. **Communication:** Communication between multiple threads is easier, as the threads share common address space. While in process we have to follow some specific communication technique for communication between two processes.

Types of Threads

There are **two** types of threads.

User Level Thread

Kernel Level Thread

What is Deadlock in OS?

All the processes in a system require some resources such as central processing unit (CPU), file storage, input/output devices, etc. to execute it. Once the execution is finished, the process releases the resource it was holding. However, when many processes run on a system they also compete for these resources they require for execution. This may arise a deadlock situation.

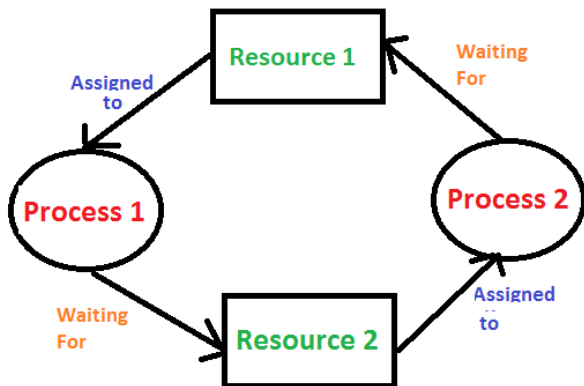
A **deadlock** is a situation in which more than one process is blocked because it is holding a resource and also requires some resource that is acquired by some other process. Therefore, none of the processes gets executed.

Necessary Conditions for Deadlock

The four necessary conditions for a deadlock to arise are as follows.

- **Mutual Exclusion:** Only one process can use a resource at any given time i.e. the resources are non-sharable.
- **Hold and wait:** A process is holding at least one resource at a time and is waiting to acquire other resources held by some other process.
- **No preemption:** The resource can be released by a process voluntarily i.e. after execution of the process.
- **Circular Wait:** A set of processes are waiting for each other in a circular fashion. For example, let's say there are a set of processes {P₀, P₁, P₂, P₃} such that P₀ depends on P₁, P₁ depends on P₂, P₂ depends on P₃

and P_{3P3} depends on P_{0P0}. This creates a circular relation between all these processes and they have to wait forever to be executed.



In the above figure, there are two processes and two resources. Process 1 holds "Resource 1" and needs "Resource 2" while Process 2 holds "Resource 2" and requires "Resource 1". This creates a situation of deadlock because none of the two processes can be executed. Since the resources are non-shareable they can only be used by one process at a time (Mutual Exclusion). Each process is holding a resource and waiting for the other process to release the resource it requires. None of the two processes releases their resources before their execution and this creates a circular wait. Therefore, all four conditions are satisfied.

Methods of Handling Deadlocks in Operating System

The first two methods are used to ensure the system never enters a deadlock.

Deadlock Prevention

This is done by restraining the ways a request can be made. Since deadlock occurs when all the above four conditions are met, we try to prevent any one of them, thus preventing a deadlock.

Deadlock Avoidance

When a process requests a resource, the deadlock avoidance algorithm examines the resource-allocation state. If allocating that resource sends the system into an unsafe state, the request is not granted.

Therefore, it requires additional information such as how many resources of each type are required by a process. If the system enters into an unsafe state, it has to take a step back to avoid deadlock.

Deadlock Detection and Recovery

We let the system fall into a deadlock and if it happens, we detect it using a detection algorithm and try to recover.

Some ways of recovery are as follows.

- Aborting all the deadlocked processes.
- Abort one process at a time until the system recovers from the deadlock.
- Resource Preemption: Resources are taken one by one from a process and assigned to higher priority processes until the deadlock is resolved.

Measuring Performance of Computer System

Throughput

Throughput is a way to find the efficiency of a CPU. It can be defined as the number of processes executed by the CPU in a given amount of time. For example, let's say, the process P1 takes 3 seconds for execution, P2 takes 5 seconds, and P3 takes 10 seconds. So, throughput, in this case, the throughput will be $(3+5+10)/3 = 18/3 = 6$ seconds.

Turnaround time

Turnaround time is the total amount of time spent by the process from coming in the ready state for the first time to its completion.

Turnaround time = Burst time + Waiting time

For **example**, if we take the First Come First Serve scheduling algorithm, and the order of arrival of processes is P1, P2, P3 and each process is taking 2, 5, 10 seconds. Then the turnaround time of P1 is 2 seconds because when it comes at 0th second, then the CPU is allocated to it and so the waiting time of P1 is 0 sec and the turnaround time will be the Burst time only

Response time

Response time is the time spent when the process is in the ready state and gets the CPU for the first time. For example, here we are using the First Come First Serve CPU scheduling algorithm for the below 3 processes:

Process	Arrival time	Burst time
P1	0 ms	8 ms
P2	1 ms	7 ms
P3	2 ms	10 ms

AfterAcademy

Here, the response time of all the 3 processes are:

- **P1:** 0 ms
- **P2:** 7 ms because the process P2 have to wait for 8 ms during the execution of P1 and then after it will get the CPU for the first time. Also, the arrival time of P2 is 1 ms. So, the response time will be $8 - 1 = 7$ ms.
- **P3:** 13 ms because the process P3 have to wait for the execution of P1 and P2 i.e. after $8 + 7 = 15$ ms, the CPU will be allocated to the process P3 for the first time. Also, the arrival of P3 is 2 ms. So, the response time for P3 will be $15 - 2 = 13$ ms.

Response time = Time at which the process gets the CPU for the first time - Arrival time

Waiting time

Waiting time is the total time spent by the process in the ready state waiting for CPU. For example, consider the arrival time of all the below 3 processes to be 0 ms, 0 ms, and 2 ms and we are using the First Come First Serve scheduling algorithm.

Process	Arrival time	Burst time
P1	0 ms	8 ms
P2	0 ms	7 ms
P3	2 ms	10 ms

Gantt Chart

P1		P2		P3	
0 ms	8 ms	8 ms	15 ms	15 ms	25 ms

AfterAcademy

Then the waiting time for all the 3 processes will be:

- **P1:** 0 ms
- **P2:** 8 ms because P2 have to wait for the complete execution of P1 and arrival time of P2 is 0 ms.
- **P3:** 13 ms becuase P3 will be executed after P1 and P2 i.e. after $8+7 = 15$ ms and the arrival time of P3 is 2 ms. So, the waiting time of P3 will be: $15-2 = 13$ ms.

Waiting time = Turnaround time - Burst time

In the above example, the processes have to wait only once. But in many other scheduling algorithms, the CPU may be allocated to the process for some time and then the process will be moved to the waiting state and again after some time, the process will get the CPU and so on.

Burst time

Every process in a computer system requires some amount of time for its execution. This time is both the CPU time and the I/O time. The CPU time is the time taken by CPU to execute the process. While the I/O time is the time taken by the process to perform some I/O operation. In general, we ignore the I/O time and we consider only the CPU time for a process. So, **Burst time is the total time taken by the process for its execution on the CPU.**